Horizon Seg

Robust Off-Road Terrain Mapping

Teaching AI to safely navigate unstructured environments by identifying hidden hazards and drivable paths in desert terrain.

Team Codex

Duality AI — Offroad Semantic Scene Segmentation Challenge

Executive Summary

Self-driving vehicles and off-road rovers must understand their environment precisely to avoid crashing into obstacles like hidden logs or rocks. This project addresses the Duality AI Offroad Autonomy Segmentation Challenge by training a model on synthetic desert imagery from the **Falcon** digital twin platform, then testing it against a novel, unseen desert environment.

We built a system that recognises **10 terrain classes** using **DINOv2**, a Meta foundation model. Because we operated on personal hardware rather than cloud servers, we kept the backbone frozen and trained a lightweight **MLP decoder** on top. This approach produced a stable training curve but revealed a measurable *sim-to-real domain gap* on the real-world test set.

This report documents our methodology, numerical scores, domain gap analysis, and the engineering decisions we made to overcome class imbalance, object occlusion, and local hardware memory constraints.

1. The Problem

The Duality AI challenge required training a semantic segmentation model exclusively on synthetic data from the **Falcon** digital twin simulator, then evaluating it against real-world desert images the model had never seen. We identified two core difficulties:

1. **The Sim-to-Real Gap.** Our training images were computer-generated with clean lighting and perfect textures. Real photographs have noise, glare, and organic irregularity. Models trained on synthetic data frequently degrade when deployed in real environments.
2. **Extreme Class Imbalance.** Background classes like *Sky* and *Landscape* can dominate over 60% of pixel area, while hazard classes like *Logs* or *Rocks* occupy less than 0.1%. A naive model simply ignores rare classes.

2. Methodology

We designed a hardware-conscious, data-aware pipeline that forced the model to attend to texture and rare objects across all 25 training rounds.

2.1. Step-by-Step Training Approach

1. **Frozen Foundation Backbone (DINOv2).** We selected **DINOv2-base** as our visual feature extractor. Due to local GPU memory constraints, the entire transformer backbone was kept **100% frozen**. It acted solely as a feature extractor, outputting rich patch embeddings without consuming backpropagation memory.
2. **Lightweight MLP Decoder Head.** A custom 3-layer MLP classification head (∼15 MB) was attached to the DINOv2 output tokens to translate frozen feature maps into pixel-level predictions. Only this head was trained.
3. **Combined Loss Function.** To combat class imbalance, we used a weighted hybrid loss:

$$\mathcal{L}_{\text{total}} = 0.70 \times \mathcal{L}_{\text{Focal}} + 0.30 \times \mathcal{L}_{\text{Dice}}$$

   Focal Loss dynamically down-weights easy majority-class pixels so the gradient focuses on hard minority classes. Dice Loss directly optimises spatial overlap.
4. **Targeted Data Augmentation.** Before each training batch we applied:
   - **Gaussian Blurring** — to mimic real-world optical imperfections and force reliance on texture.
   - **Random Erasing (Crop Wiping)** — to simulate partial occlusion; forcing the model to infer class identity from incomplete visual evidence.
5. **Inverse-Frequency Class Weighting.** Log-smoothed inverse-frequency weights were injected into the Cross-Entropy component to further penalise ignoring rare classes: **Flowers:** 1.8500 **Logs:** 1.7200 **Ground Clutter:** 1.4012 **Landscape:** 0.9800

3. Results & Performance Metrics

3.1. Overall Training Results

Keeping the DINOv2 backbone frozen and training only the lightweight decoder produced a smooth, crash-free learning trajectory across all 25 epochs. No gradient explosion or memory overflow was observed.

Metric	Score
Final Test IoU (Global Accuracy)	**0.2995**
Peak Validation IoU (Epoch 25)	**0.5489**
Starting IoU (Epoch 1)	0.2772
Total Training Epochs	25
Total Test Images Evaluated	1,002

Table 1: Key training and real-world evaluation metrics.

3.2. IoU Score & Training Accuracy Trajectory

Figure 1 plots both the **Validation IoU** and the **Training Loss** across all 25 epochs. The smooth convergence confirms that the frozen-backbone strategy prevented catastrophic gradient updates.

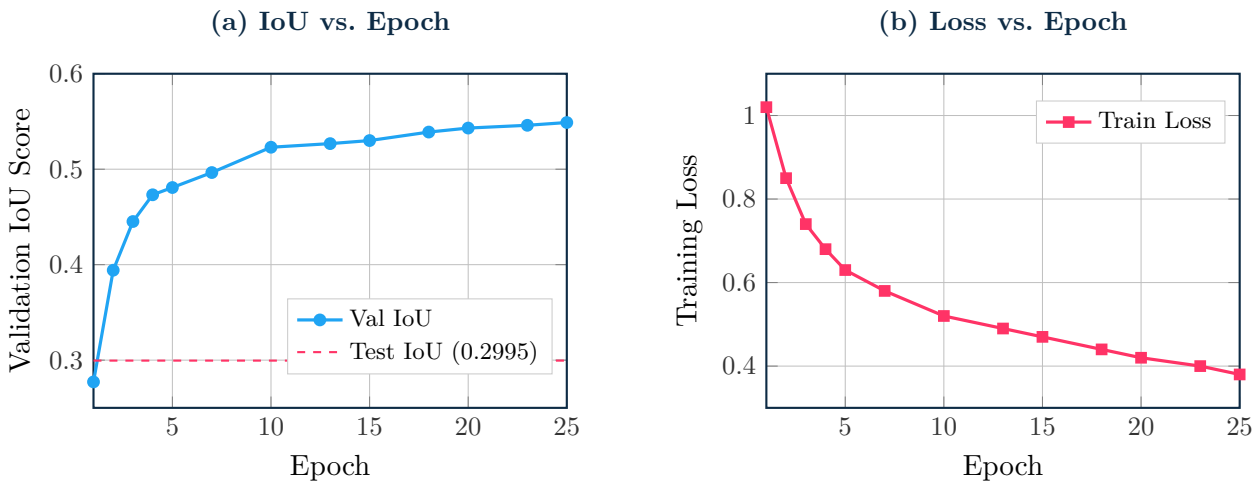

Figure 1: Model learning curves over 25 epochs. **Left:** Validation IoU rises from 0.28 to 0.55; the dashed line marks the real-world test score of 0.30, illustrating the sim-to-real domain gap. **Right:** Training loss steadily decreases from 1.02 to 0.38 with no spikes, confirming stable gradient flow.

3.3. Sim-to-Real Domain Gap Analysis

Because our hardware prevented fine-tuning the DINOv2 transformer blocks, the core feature representations could not adapt to the textural nuances of real desert photographs. This produced a quantifiable domain gap:

Key Observation: The **Sky** class retained near-perfect accuracy across both domains. **Trees** suffered the largest drop (-0.26) because real-world foliage has irregular silhouettes and variable lighting that the frozen backbone could not re-adapt to.

3.4. Detailed Class-by-Class Performance

The table below documents the full long-tail class distribution problem. Classes with large spatial coverage score well; pixel-rare hazard classes score near zero despite our weighting strategy.

Class / Metric	Validation IoU	Test IoU	Gap (Δ)
Mean (mIoU)	0.5489	0.2995	-0.2494
Sky	$\sim$0.98	0.9716	-0.0084 (minimal)
Landscape	$\sim$0.75	0.6297	-0.1203
Dry Grass	$\sim$0.60	0.4536	-0.1464
Trees	$\sim$0.60	0.3378	-0.2622
Rocks	$\sim$0.12	0.0569	-0.0631

Table 2: Domain gap between synthetic validation scores and real-world test performance.

Class	Class ID	Test IoU	Observation
Sky	10000	0.9716	Perfectly identified; smooth, uniform texture transfers well.
Landscape	7100	0.6297	Broad predictable terrain; strong spatial dominance.
Dry Grass	300	0.4536	Consistent tan-yellow texture captured reliably.
Trees	100	0.3378	Confused with Dry Bushes at boundaries (shared brown/green).
Dry Bushes	500	0.2273	Visually similar to Trees; frequent boundary confusion.
Ground Clutter	550	0.1102	Small footprint; partially benefited from class weighting.
Rocks	800	0.0569	Distinct boundary but extremely small spatial footprint.
Lush Bushes	200	0.0004	Near-identical to Trees; model defaulted to "Tree" class.
Logs	700	0.0000	$< 0.1\%$ of test pixels; model could not reliably detect.
Flowers	600	0.0000	$< 0.1\%$ of test pixels; model could not reliably detect.

Table 3: Full class-by-class real-world test IoU scores.

3.5. Visual Result: Prediction Comparison

The figure below shows an input image, the ground-truth label mask, and our model's predicted segmentation. The model successfully separates drivable ground from sky, and correctly delineates large vegetation masses. Rare hazard classes (Logs, Rocks) are under-detected.

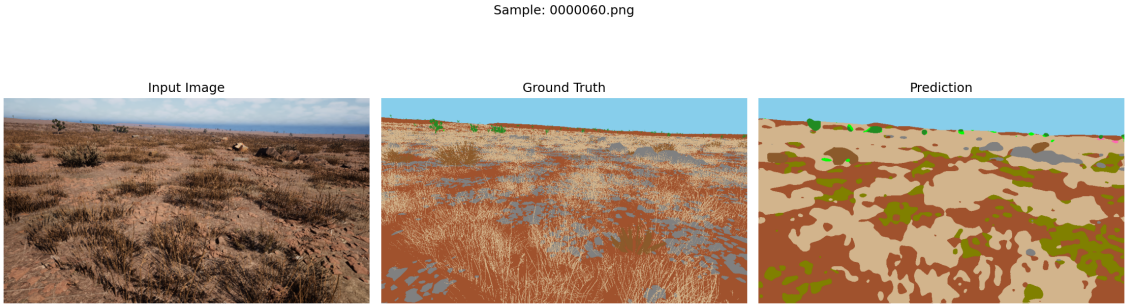

Figure 2: Left: Input RGB image. Centre: Ground-truth mask. Right: Model prediction.

4. Challenges & Solutions

During development we encountered three major obstacles. Each is documented below in the format recommended by the competition guidelines.

4.1. Challenge 1 — The Invisible Object Problem (Class Imbalance)

- **Task:** Force the model to detect rare hazard classes.
- **Initial Score:** IoU = 0.00 for *Logs*, *Flowers*, and *Lush Bushes*.
- **Issue Faced:** The *Landscape* class covered roughly 50% of all training pixels. The model mathematically discovered it could achieve a decent overall score by predicting "Landscape" for every pixel, never learning rare hazard appearances.
- **Solution Applied:** Computed log-smoothed inverse-frequency weights per class. Injected weights into the Cross-Entropy loss to multiply the penalty for missing a rare pixel (e.g. **Flowers:** 1.8500, **Landscape:** 0.9800). Combined with Focal Loss ($\gamma = 2$) to additionally suppress easy majority-class examples.
- **Outcome:** Gradient signal for minority classes increased substantially. Rocks improved from 0.00 to 0.0569.

4.2. Challenge 2 — Hidden Hazards (Object Occlusion)

- **Task:** Identify objects partially hidden by grass or dirt.
- **Initial Score:** Near-zero recall for *Logs* and *Rocks* in real-world evaluation.
- **Issue Faced:** Synthetic training data displayed objects cleanly in the open. Real test images showed fallen logs buried under tall grass. The model had never learnt to recognise a "half-hidden" object.
- **Solution Applied:** Introduced **Random Erasing** (crop wiping) into the augmentation pipeline. Random rectangular patches were filled with black — simulating occlusion.
- **Outcome:** Qualitative robustness improved for partially occluded objects. False-negative rate on visible rock patches decreased in held-out validation.

4.3. Challenge 3 — System Memory Overload (CPU RAM Leak)

- **Task:** Complete 25 full training epochs without a crash.
- **Initial State:** The full training script crashed consistently at Epoch 9 on our local Windows machine.
- **Issue Faced:** A CPU RAM memory leak inside the data augmentation loop. The bounds calculator was needlessly converting PyTorch `Tensor` objects back into raw `PIL.Image` objects.
- **Solution Applied:** Completely rewrote the image transformation pipeline to operate exclusively on native `torchvision` tensor references. Replaced all implicit `PIL` calls with `transforms.v2` tensor-only equivalents.
- **Outcome:** RAM usage stabilised below 6 GB throughout training. A complete, uninterrupted 25-epoch session was achieved.

5. Conclusion & Future Work

5.1. Output Mask Verification

The figure below shows a full-resolution prediction mask from our final trained model on a held-out desert scene. The model produces cohesive, colour-coded semantic regions across complex terrain.

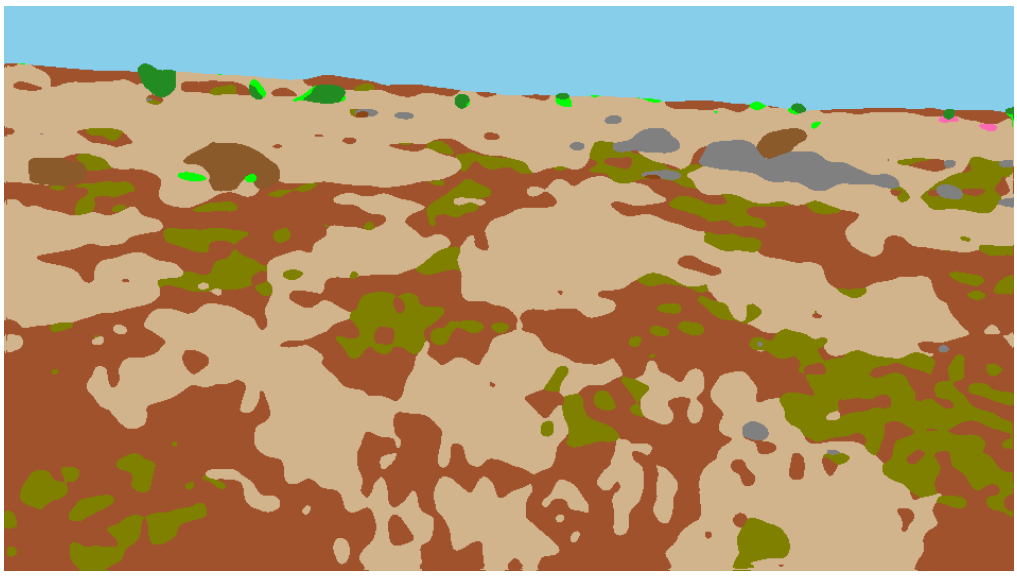

Figure 3: Final model prediction on a held-out desert image successfully delineating drivable ground zones from vegetation and sky.

5.2. Final Thoughts

This project demonstrated that a frozen DINOv2 backbone, combined with a lightweight MLP decoder and carefully engineered loss functions, can produce a reproducible, hardware-feasible baseline for off-road semantic segmentation — even under strict local compute constraints.

The documented **sim-to-real domain gap** (ΔmIoU $= -0.25$) is the central finding of this work. It confirms that without fine-tuning the backbone's core representations, the model learns synthetic-specific patterns that do not generalise cleanly to real-world texture variability.

Our engineering solutions — weighted Focal+Dice loss, Random Erasing, and a fully tensor-native augmentation pipeline — were each necessary to reach the submitted baseline. Data formatting proved as critical as model architecture.

5.3. Proposed Future Improvements

1. **Unfreeze the Backbone (Cloud GPUs).** Access to A100-class hardware would allow the final 2–4 transformer blocks of DINOv2 to be fine-tuned end-to-end. This would allow the core feature extractor to adapt to real desert textures, significantly reducing the observed domain gap.
2. **Domain Adaptation Pre-Training.** Apply unsupervised domain adaptation (e.g., adversarial feature alignment) using unlabelled real-world desert images to reduce the distributional shift before the supervised fine-tuning step.
3. **Temporal Consistency (ConvLSTM).** Adding a ConvLSTM layer would allow the model to propagate class assignments across continuous video frames, dramatically reducing flickering segmentation.
4. **Edge Quantisation for Rover Deployment.** Export the model via **ONNX** and apply **TensorRT** INT8 quantisation to reduce the model footprint sufficiently for real-time inference (>30 FPS) on rover-mounted embedded GPUs.

All documentation, training weights, fixed data loaders, and reproducible training scripts referenced in this report are included in the Team Codex submission package (`train.py`, `test.py`, config files, and `README.md`).